

Problems on Higher-Order Functions

Each problem includes a sample input and expected output for better understanding.

SECTION A - BASIC

Core syntax, built-in operations, and straightforward practice.

1. Write a Python higher-order function that accepts another function and two numbers, then applies that function to the numbers.

Input	operation = add a = 10 b = 20
Expected Output	Result = 30

2. Write a Python program to double every number in a list using map().

Input	numbers = [1, 2, 3, 4, 5]
Expected Output	[2, 4, 6, 8, 10]

3. Write a Python program to square every number in a list using map().

Input	numbers = [1, 2, 3, 4]
Expected Output	[1, 4, 9, 16]

4. Write a Python program to select even numbers from a list using filter().

Input	numbers = [1, 2, 3, 4, 5, 6]
Expected Output	[2, 4, 6]

5. Write a Python program to select only positive values using filter().

Input	numbers = [-5, 3, 0, -2, 7, 9]
Expected Output	[3, 7, 9]

6. Write a Python program to sort words by length by passing a key function to sorted().

Input	words = ['python', 'AI', 'data', 'cloud']
Expected Output	['AI', 'data', 'cloud', 'python']

7. Write a Python program to calculate the sum of a list using functools.reduce().

Input	numbers = [10, 20, 30, 40]
Expected Output	Sum = 100

Problems on Higher-Order Functions

Each problem includes a sample input and expected output for better understanding.

8. Write a Python program to calculate the product of a list using `functools.reduce()`.

Input	numbers = [2, 3, 4, 5]
Expected Output	Product = 120

9. Write a Python function that returns another function for multiplying a value by a fixed number.

Input	Create multiplier for 5 Input value = 8
Expected Output	40

10. Write a Python function that returns a greeting function based on the selected language.

Input	language = Marathi name = Asha
Expected Output	Namaskar, Asha!

11. Write a Python higher-order function that accepts a transformation function and applies it to every item in a list.

Input	data = [1, 2, 3] transformation = cube
Expected Output	[1, 8, 27]

12. Write a Python function that returns add, subtract, multiply, or divide function according to an operation name.

Input	operation = 'subtract' a = 30 b = 12
Expected Output	Result = 18

13. Write a Python program to filter names having at least five characters by passing a predicate function.

Input	names = ['Asha', 'Rohan', 'Neha', 'Sameer']
Expected Output	['Rohan', 'Sameer']

14. Write a Python program to convert every word to uppercase using `map()` with `str.upper`.

Input	words = ['python', 'ai', 'data']
Expected Output	['PYTHON', 'AI', 'DATA']

Problems on Higher-Order Functions

Each problem includes a sample input and expected output for better understanding.

15. Write a Python program to sort tuples by their second element using a key function.

Input	items = [(1, 50), (2, 20), (3, 40)]
Expected Output	[(2, 20), (3, 40), (1, 50)]

16. Write a Python higher-order function that calls a supplied function a given number of times and collects the returned values.

Input	function returns Hello repeat = 3
Expected Output	['Hello', 'Hello', 'Hello']

Problems on Higher-Order Functions

Each problem includes a sample input and expected output for better understanding.

SECTION B - INTERMEDIATE

Multi-step logic and practical manipulation of the topic.

1. Write a Python higher-order function to compose two functions so that the output of the first becomes the input of the second.

Input	first = add 3 second = square input = 5
Expected Output	64

2. Write a Python program to build a transformation pipeline that strips spaces, converts text to lowercase, and replaces spaces with hyphens.

Input	text = " Machine Learning "
Expected Output	machine-learning

3. Write a Python program to first filter even numbers and then square them using higher-order functions.

Input	numbers = [1, 2, 3, 4, 5, 6]
Expected Output	[4, 16, 36]

4. Write a Python program to map numbers to their squares and then reduce them to a total.

Input	numbers = [1, 2, 3, 4]
Expected Output	Sum of squares = 30

5. Write a Python function factory that returns a function for raising numbers to a fixed power.

Input	Create power function for exponent 3 Input = 4
Expected Output	64

6. Write a Python function that returns a counter function. Each call to the returned function should increase the count.

Input	Call returned counter 4 times
Expected Output	1, 2, 3, 4

Problems on Higher-Order Functions

Each problem includes a sample input and expected output for better understanding.

7. Write a Python function that sorts student dictionaries using a key function passed as an argument.

Input	students = [{'name':'Asha','score':82}, {'name':'Ravi','score':74}, {'name':'Neha','score':91}] key = score
Expected Output	['Ravi', 'Asha', 'Neha']

8. Write a generic Python aggregator function that accepts data and an aggregation function.

Input	data = [10, 20, 30, 40] aggregation = maximum
Expected Output	40

9. Write a Python higher-order function that applies a list of functions to the same input value.

Input	functions = [square, cube, double] input = 3
Expected Output	[9, 27, 6]

10. Write a Python function factory that creates a validator for a specified minimum and maximum range.

Input	Create validator for 18 to 60 value = 25
Expected Output	Valid = True

Problems on Higher-Order Functions

Each problem includes a sample input and expected output for better understanding.

SECTION C - ADVANCED

Algorithmic problem solving and deeper application of the topic.

1. Write a curried Python function where adding three values is performed through three successive function calls.

Input	<code>add(10)(20)(30)</code>
Expected Output	60

2. Write a Python function that accepts a list of functions and builds a reusable processing pipeline.

Input	<code>pipeline = [double, add_one, square]</code> <code>input = 3</code>
Expected Output	49

3. Write your own higher-order `my_map()` function without using Python's built-in `map()`.

Input	<code>function = square</code> <code>data = [1, 2, 3, 4]</code>
Expected Output	[1, 4, 9, 16]

4. Write your own higher-order `my_filter()` function without using Python's built-in `filter()`.

Input	<code>predicate = is_even</code> <code>data = [1, 2, 3, 4, 5, 6]</code>
Expected Output	[2, 4, 6]

5. Write your own higher-order `my_reduce()` function without using `functools.reduce()`.

Input	<code>function = add</code> <code>data = [10, 20, 30, 40]</code>
Expected Output	100

6. Write a higher-order memoize function that wraps a function and caches previously calculated results.

Input	Wrapped function = <code>square</code> Calls = <code>square(5)</code> , <code>square(5)</code> , <code>square(7)</code>
Expected Output	Results = 25, 25, 49 Actual calculations = 2

Problems on Higher-Order Functions

Each problem includes a sample input and expected output for better understanding.

7. Write a higher-order retry function that calls another function again when it fails, up to a given retry limit.

Input	Function fails twice, succeeds third time retries = 3
Expected Output	Success on attempt 3

8. Write a function factory that creates different discount policies and returns the appropriate pricing function.

Input	policy = premium price = 2000 premium discount = 15%
Expected Output	Final price = 1700.0

9. Write a callback-based event handler that registers multiple functions and executes all callbacks when an event occurs.

Input	callbacks = [send_email, write_log] event = OrderPlaced
Expected Output	send_email executed write_log executed

10. Write a strategy-based higher-order function that selects a processing function at runtime according to a mode.

Input	mode = 'normalize' data = [10, 20, 30]
Expected Output	[0.0, 0.5, 1.0]